

[Hi, I'm Hwee-Boon Yar](#)[About](#)

Using the Skill-Creator Skill to Improve Your Existing Skills

Mar 3, 2026

Anthropic's [skill-creator](#) skill recently got an update that adds evaluation capabilities — the ability to test your skill against synthetic prompts, grade the outputs, and compare old vs new versions. I ran it on my `writing-voice` skill to see how it works.

What the Skill-Creator Does Now

The skill-creator is a meta-skill for creating and improving other skills. It could already help you write new skills and edit existing ones. The update adds evaluation capabilities — so now it can also:

- Analyze your existing skill against real examples (blog posts, code, whatever it's supposed to produce)
- Generate test cases with assertions
- Run those tests against both the old and new versions of the skill in parallel
- Grade the outputs and show you a comparison in a browser-based viewer
- Optionally optimize the skill's description so the agent triggers it more reliably

The [commit](#) adds a bunch of supporting infrastructure — subagent prompts for analysis, comparison, and grading, Python scripts for running evals and generating reports, and an HTML viewer for reviewing results.

Trying It on My Writing-Voice Skill

I have a `writing-voice` skill — a markdown file that tells the agent how to write in my voice. Other skills can call it when they need to match my tone. It triggers on phrases like “use my voice” or “write in my style.” I wrote about [how skills fit into my AI coding workflow](#) previously.

The v1.0.0 skill covered voice, structure, tone, formatting, and what to avoid, all extracted from my actual published writing. It worked, but I wanted to see if the skill-creator could find gaps.

Studying My Posts

The agent fetched 5 of my blog posts and analyzed patterns the existing skill didn't capture. It found quite a few:

- **Problem-solution arc** — nearly every post follows: problem, what I built, how it works, the result. The old skill mentioned this loosely but wasn't specific.
- **Opener patterns** — I use distinct opener types: problem/situation, question from a reader, observation, reference to prior work. Not documented.
- **Closing patterns** — I often just stop after the last point. Sometimes a practical final line. Never a summary paragraph.
- **“The short answer: X” pattern** — a quick summary right after the opener.
- **Contrast/reframing** — “The question isn't really A vs B. It's C.”
- **Sentence fragments for emphasis** — “Fair enough.” “That's it.”
- **Matter-of-fact vs combative** — the old skill said “opinionated” but my actual voice is more matter-of-fact. I state what works for me without attacking alternatives.
- **Short paragraphs** — rarely more than 3-4 sentences. Often 1-2.

The agent wrote a v2.0.0 incorporating all of this.

Running Evals

The skill-creator generated 3 test prompts designed to exercise different aspects of the voice:

- 1. A Render-to-Hetzner migration post — tests cost comparisons, acknowledging alternatives, specific numbers
- 2. A Bun OG image script post — tests the short "here's a script I wrote" format, problem-solution arc
- 3. A Drizzle vs Prisma comparison post — tests fair-comparison tone, reframing pattern

If this resonates, [Stacknaut](#) is where I put everything I learn into practice — a SaaS starter kit that reflects how I actually build and ship.

For each prompt, two subagents ran in parallel — one with the old skill, one with the new. That's 6 runs total. Each output was graded against assertions like "uses first person throughout," "no inflating adverbs," "short paragraphs," "acknowledges alternatives fairly."

Results

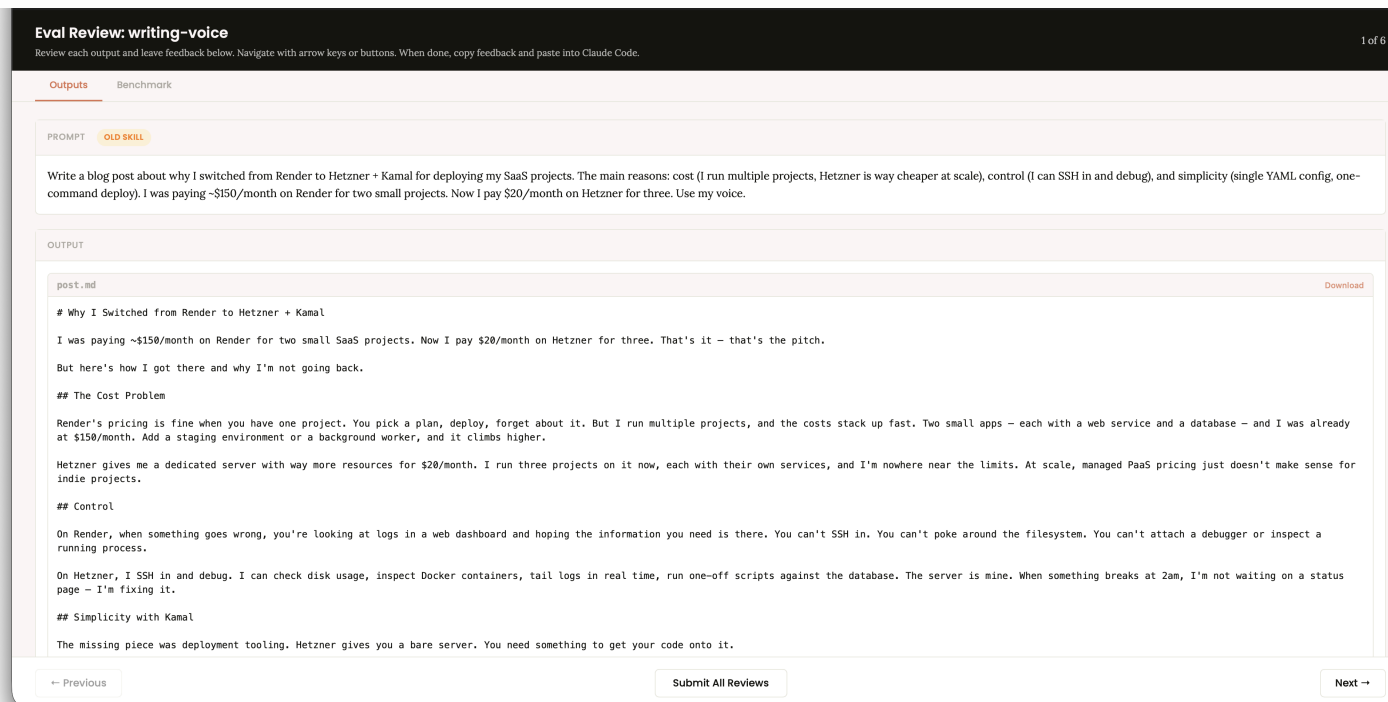
Test Case	New Skill	Old Skill
Render to Hetzner	10/10 (100%)	9/10 (90%)
Bun OG Script	9/9 (100%)	9/9 (100%)
Drizzle vs Prisma	9/10 (90%)	9/10 (90%)

The quantitative scores were close. That makes sense for a writing voice skill — the real differences are qualitative. Reading both outputs side by side, the new skill produced writing that felt closer to

how I actually write. The old skill's output was fine but missed some of the patterns — no reframing, longer paragraphs, more generic closings.

The Eval Viewer

The skill-creator generated a static HTML viewer that opened in the browser with two tabs — one for reading the full outputs and leaving feedback, one for the benchmark numbers. You click through each output one at a time — old skill for test case 1, new skill for test case 1, old skill for test case 2, and so on. Not truly side by side, but easy enough to compare when you've just read the previous version. And if it bothers you enough — well, you do have a coding agent right there.



I clicked through all 6, submitted my review — which downloads a `feedback.json` that the agent reads back.

Downloads

feedback.json838 bytes

Clear

Eval Review: writing-voice

Review each output and leave feedback below. Navigate w

and paste into Claude Code.

2 of 6

Outputs

Benchmark

PROMPT

WITH SKILL

Write a blog post about why I switched from Render to Hetzner + Kamal for deploying my SaaS projects. The main reasons: cost (I run multiple projects, Hetzner is way cheaper at scale), control (I can SSH in and debug), and simplicity (single YAML config, one-command deploy). I was paying ~\$150/month on Render for two small projects. Now I pay \$20/month on Hetzner for three. Use my voice.

OUTPUT

post.md

Download

Why I Switched from

I was paying ~\$150/mo
That's the whole story

The Problem with Re

Render is fine. It works. You push code, it deploys. For a single project with a simple setup, it's a reasonable choice.

But I run multiple projects, and costs add up fast. Each service, each database, each background worker – they all have their own line item. Two small projects with a web service, a background worker, and a managed Postgres each, and you're looking at \$150/month before you've done anything interesting.

The other issue: when something goes wrong, you're stuck with their dashboard and logs viewer. No SSH. No poking around the filesystem. No running a quick query against the database to figure out what happened. You're debugging through a web UI, which is fine until it isn't.

Why Hetzner + Kamal

I switched to a single Hetzner VPS running all three projects, deployed with Kamal.

Cost. A Hetzner CX32 – 4 vCPUs, 8GB RAM – is about \$7/month. Add a managed Postgres instance and

Review Complete

Your feedback has been saved. Go back to your Claude Code session and tell Claude you're done reviewing.

OK

← Previous

Submit All Reviews

Next →

Both versions were fine, but the improved one was noticeably better on the subtleties.

Was It Worth It?

The whole process took one session. The agent did the heavy lifting — reading my posts, identifying gaps, writing the improved skill, generating tests, running them, and producing the comparison viewer. I just reviewed the outputs and confirmed.

For a style skill like `writing-voice`, the evaluation framework is probably overkill — you can tell by reading two paragraphs whether the voice is right. But for more complex skills with specific behavioral requirements, being able to run assertions against outputs and compare versions systematically is useful. The skill-creator now gives you that without having to build your own eval harness.

I need to try this out more. The eval process is designed to loop — improve, test, review, improve again — so you can keep tightening a skill over multiple iterations. I can see this being genuinely useful for people who build and share skills, or even sell them. Whether most people will actually take the time to run evals diligently is another question. But the infrastructure is there now, and it's good.

Ship a profitable SaaS on one server

Everything I learn goes into this starter kit — I extracted my SaaS work from 30 years of programming experience into a solid starter kit — one monorepo, one server, one-command deploy. Designed to work with AI coding agents.

[Check out Stacknaut](#)

Launch discount available.

Hi, I'm Hwee-Boon Yar
hboon@motionobj.com

 [hboon](#)
 [hboon](#)
 [hboon](#)

I write, ship and sell software products. Indie.
Writing code in Swift, TypeScript and Ruby.
Based in Singapore, working remotely. This is
my blog.